Extract Insights from Visual Data on Azure

Vision, image & video generation, content understanding, and search

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

© Rick Beerendonk

What You'll Learn

- Send an image to a model and ask questions about it
- Generate images and video, and edit part of an image
- Pull named fields out of forms, invoices, and mixed media
- Choose between a model, an analyzer, and a document model
- Turn a pile of files into something searchable

© Rick Beerendonk

Problem: 90% of Company Content Is Not a Database Row

Scanned invoices. Photos from the field. Recorded meetings. Slide decks in a share.

- ❌ You cannot query a PDF
- ❌ You cannot filter a photo
- ❌ Someone retypes the invoice total into the finance system by hand

Need: turn pixels and pages into fields, text, and searchable records.

This presentation is four different tools for that one job — and knowing which to reach for is most of the skill.

glasspaper oblicum

© Rick Beerendonk

Which Tool for Which Job?

```
What do you have, and what do you want back?

  "Describe / reason about this image"          ⟶ multimodal model  (gpt-5.2)
       open-ended answer, no fixed schema

  "Give me these exact fields"                  ⟶ Document Intelligence
       invoice total, IBAN, signature box          forms and documents

  "Same, but the input might be audio,          ⟶ Content Understanding
  video, or an image too"                          one analyzer, many media types

  "Make me a picture / a video"                 ⟶ gpt-image-2 / FLUX / Sora

  "Let people search 100,000 of these"          ⟶ AI Search + skillset
```

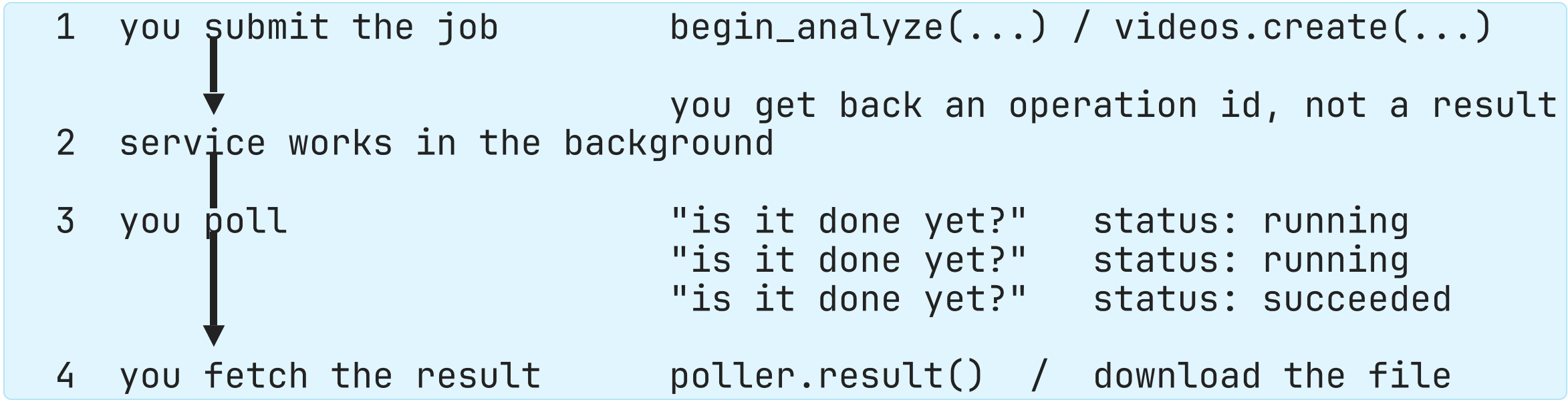
The dividing line: a *model* gives you an answer; an *analyzer* gives you the same fields every time.

glasspaper oblicum

© Rick Beerendonk

Everything Big Is Asynchronous

Image analysis, video generation, and document analysis all take longer than one HTTP request.



That is why the SDK calls start with `begin_` and return a *poller*. When you see `Operation-Location` in a response header, you are in step 3.

glasspaper oblicum

© Rick Beerendonk

Vision-Enabled Generative AI

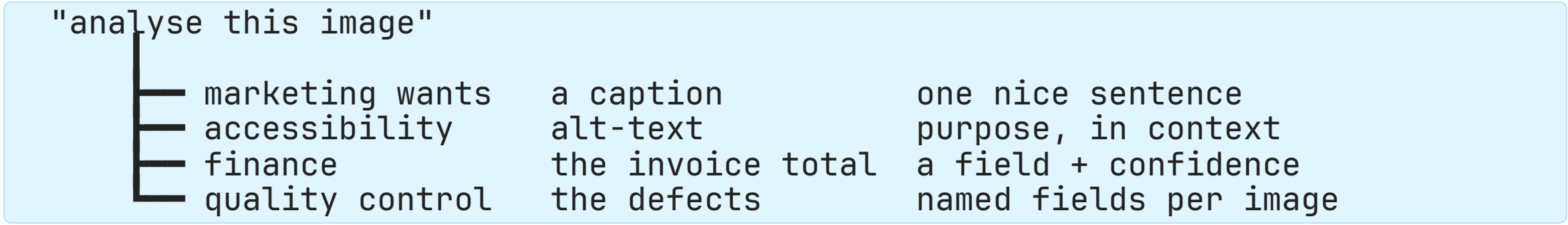
glasspaper

oblicum

© Rick Beerendonk

Problem: Everyone Says "Analyse the Image"

Four people say the same sentence and mean four different things.



Need: name the required *output* first. The service then follows almost automatically.

Picking a service before naming the output is the most common mistake in this area — on the exam and in practice.

glasspaper oblicum

© Rick Beerendonk

Prose or Evidence?

The deepest split in this topic is not "which product", it is **what the answer is made of**.

```
PROSE                                  EVIDENCE
a model describes                      an extractor locates

"The invoice total is 1,240 EUR"       total: 1240.00
                                       confidence: 0.94
                                       page 2, polygon [x,y,...]

great for humans                       required for automation,
cannot be verified automatically       review queues, and audits
```

Ask: will a human read this, or will a system act on it? Only the second needs coordinates, confidence, and citations.

glasspaper oblicum

© Rick Beerendonk

Visual Understanding Patterns

[EXAM]

A multimodal solution may need several distinct outputs:

Output	Example
Caption	“A delivery truck is parked beside a loading dock.”
Alt-text	A concise description usable by a screen reader
Visual question answering	“What color is the truck?”
Grounded extraction	Field value plus its location on the page
Object identification	Objects, components, or defects as named fields
Video understanding	Events, timestamps, speakers, or segment summaries

No separate vision service is needed: a multimodal model writes the first three outputs from a prompt; a Content Understanding analyzer returns the last three as fields.

© Rick Beerendonk

Multimodal Models

Model	Provider	Notes
`gpt-5.2`	OpenAI	Supports Responses API and ChatCompletions
`gpt-5-mini`	OpenAI	Lighter, cost-effective variant
`Phi-4-multimodal-instruct`	Microsoft	ChatCompletions API only

Image input formats: web URL or Base64 data URL: `{data}`

Limits: up to 10 images per chat request; JPEG, PNG, WEBP, or GIF (first frame only); a web URL must be publicly reachable

© Rick Beerendonk

Responses API (gpt-5.2, gpt-5-mini)

[EXAM]

```python
from openai import OpenAI
import base64

client = OpenAI(base_url=f"{ENDPOINT}/openai/v1/", api_key=KEY)

with open("image.jpg", "rb") as f:
    b64 = base64.b64encode(f.read()).decode("utf-8")

response = client.responses.create(
    model=DEPLOYMENT,
    input=[
        {"role": "developer", "content": "You analyze images."},
        {
            "role": "user",
            "content": [
                {"type": "input_text", "text": "What is in this image?"},
                {"type": "input_image", "image_url": f"{b64}"},
            ],
        },
    ],
)
print(response.output_text)
```

Note: instructions use role `"developer"`; the image goes in the `"user"` message as type `"input_image"`

glasspaper

oblicum

© Rick Beerendonk

ChatCompletions API (gpt-5.2, Phi-4)

[EXAM]

```
response = client.chat.completions.create(
    model=DEPLOYMENT,
    messages=[
        {"role": "system", "content": "You analyze images."},
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "What is in this image?"},
                {"type": "image_url",
                 "image_url": {"url": f"{b64}"}},
            ],
        },
    ],
)
print(response.choices[0].message.content)
```

Note: role is `"system"`; image type is `"image_url"` (nested `{"url": ...}`)

Multimodal request shape: the image belongs in the **user message content array** as an `image_url` item, alongside the `text` item. Base64-encoding it into the content **string**, putting the URL in request metadata, or placing it in the system message are all rejected.

Phi-4-multimodal-instruct — ChatCompletions API only; does not support Responses API

glasspaper oblicum

© Rick Beerendonk

Accessibility and Alt-Text

[EXAM]

Alt-text is not the same as a marketing caption.

Good alt-text should:

- Convey the purpose or information of the image
- Be concise and specific
- Identify relevant text, people, objects, and relationships
- Avoid guessing sensitive attributes
- Avoid “image of” unless it improves clarity
- Respect the surrounding page context

Example:

```
Poor: An image.
Caption: A modern office with people working.
Alt-text: Three people review a dashboard beside a wall display showing monthly sales.
```

Validate generated descriptions: a model can omit important visual information or invent details. Accessibility review remains part of the application design.

In the Microsoft Learn path: accessibility appears in the responsible-AI material; alt-text as a visual workflow has no lesson of its own.

© Rick Beerendonk

Retrieval Practice — Lab 1

Pause here and complete Lab 1, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Image Generation

glasspaper

oblicum

© Rick Beerendonk

Image Generation — gpt-image-2 and FLUX

[EXAM]

Models:

Model	Provider
`gpt-image-2`	OpenAI
`FLUX.1-Kontext-pro`	Black Forest Labs — same call on its own endpoint, returns `data[0].url`

```python
import base64
from openai import OpenAI

client = OpenAI(base_url=f"{ENDPOINT}/openai/v1/", api_key=KEY)

img_results = client.images.generate(
    model=MODEL_DEPLOYMENT,
    prompt="A golden retriever running through autumn leaves, photorealistic",
    n=1,
    size="1024x1024",
)

image_bytes = base64.b64decode(img_results.data[0].b64_json)
with open("output.png", "wb") as f:
    f.write(image_bytes)
```

Response: `data[0].b64_json` — GPT-image models always return base64; decode to get raw bytes

glasspaper
obliqum
© Rick Beerendonk

Image Editing — Inpainting with a Mask

[EXAM]

Use case: an existing image contains something to remove (competitor logo in the background, unwanted object) — edit only that region instead of regenerating the whole image.

API: `client.images.edit()` (not `generate`) — supplies the source image plus a **mask PNG** where transparent pixels mark the region to be re-painted.

```
result = client.images.edit(
    model="gpt-image-2",
    image=open("product-photo.png", "rb"),
    mask=open("mask.png", "rb"),        # transparent region = paint here; opaque = keep
    prompt="Replace the logo with a plain wooden wall, matching the existing lighting",
    size="1024x1024",
)
image_bytes = base64.b64decode(result.data[0].b64_json)
```

Mask rules: same dimensions as source; PNG with alpha channel; transparent pixels mark the area to repaint.

Not the same as `generate`: `generate` makes a new image; `edit` starts from the original and repaints the masked area.

© Rick Beerendonk

Keeping a Real Product Recognisable

[EXAM]

Generating a marketing scene around a real product usually distorts it — the label, shape, or colour drifts. Raise `input_fidelity` to preserve the reference:

```
result = client.images.edit(
    model="gpt-image-1.5",
    image=open("product.png", "rb"),
    prompt="Place the bottle on a marble kitchen counter, morning light",
    input_fidelity="high",     # keep the product identity
)
```

Setting	Effect
`input_fidelity="high"`	Faces and distinctive product details stay close to the input
Default (`low`)	More creative freedom, weaker identity preservation

Model matters: `input_fidelity` is a setting of `gpt-image-1.5`; with `gpt-image-2`, omit it.

Not a substitute: a groundedness filter checks text claims, and prompt wording alone does not lock appearance.

© Rick Beerendonk

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Video Generation

glasspaper



© Rick Beerendonk

Sora Video Generation

[EXAM]

Model: `sora-2`

Sizes: `"1280x720"` (landscape) or `"720x1280"` (portrait)
Durations: `"4"`, `"8"`, or `"12"` (seconds, as string value)
Concurrency limit: max 2 jobs at a time

Pattern: Create → Poll → Download

```
video = client.videos.create(
    model="sora-2",
    prompt="Ocean waves at sunrise, cinematic slow motion",
    size="1280x720",
    seconds="8",
)

# Poll until done
while video.status not in ["completed", "failed", "cancelled"]:
    video = client.videos.retrieve(video.id)

# Download
content = client.videos.download_content(video.id, variant="video")
content.write_to_file("output.mp4")
```

Expiry: Completed videos expire after **24 hours**

glasspaper oblicum

© Rick Beerendonk

Sora — Remix and Reference Image

Video remix (modify an existing video):

```
remixed = client.videos.remix(
    video_id=original_video.id,
    prompt="Change the setting to winter, keep the same scene composition",
)
```

Reference image (constrain the generated scene):

```
video = client.videos.create(
    model="sora-2",
    prompt="A cat walks through this landscape",
    size="1280x720",
    seconds="8",
    input_reference=open("landscape.png", "rb"),
)
```

Constraint: Reference images must not contain human faces

© Rick Beerendonk

Editing Video Without Regenerating It

[EXAM]

A finished clip needs one targeted change — an unwanted object, a different colour palette. Regenerating produces a **different video** — instead, **remix** the completed video: pass its ID and a prompt that describes only the change.

Approach	Result
Remix (`client.videos.remix`)	Applies the change, keeps structure, motion, and framing
Change the prompt and re-run	New generation, so the approved scene is lost
Crop the frame	Removes wanted content and changes the aspect ratio

Remix works on videos the model generated: the original must still exist, so remix within its 24 hours. The remix is a new job — poll and download it like any other.

Rule: to change *part* of an image, edit with a mask. To change *part* of a generated video, remix it. To change *the whole* asset, generate again.

© Rick Beerendonk

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Responsible AI for Visual Content

Moderate uploads, catch hidden instructions, and enforce visual policy rules.

glasspaper oblicum

© Rick Beerendonk

Moderating Uploaded Images

[EXAM]

Users upload photos to an agent. The requirement is to **classify harmful visual content and block by severity** — that is **image moderation** in Azure AI Content Safety.

```
from azure.ai.contentsafety import ContentSafetyClient
from azure.ai.contentsafety.models import AnalyzeImageOptions, ImageData

request = AnalyzeImageOptions(image=ImageData(content=image_bytes))
response = client.analyze_image(request)   # severity per harm category
```

Control	What it inspects
Image moderation	The **picture itself** — harm category and severity
OCR keyword scanning	Only text extracted from the image
Prompt shields	Injected instructions, not visual harm
Blocklists	Configured phrases, reactive and text-only

Two different image risks: moderation stops a harmful *picture*; prompt shields stop *hidden instructions* inside it, once OCR has extracted the text. A solution that accepts uploads usually needs both.

© Rick Beerendonk

Content Safety: Which Check Covers What

Check	Covers
Harm categories	Hate, sexual, violence, and self-harm — each with a severity
Direct prompt attack	Instructions in the user's own input that try to override the rules
Indirect prompt attack	Instructions hidden in documents or retrieved content
Protected material detection	Copyrighted or otherwise protected text and code

Text in an image: Prompt Shields work on text. Extract the text with OCR, then check it as a document with the document attack shield.

Order matters: check visual content before it reaches the model, so unsafe input is blocked first.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Visual Safety and Policy Controls

[EXAM]

Safety must cover both the prompt and the visual content.

Risk	Possible control
Unsafe generated image	Content filters, moderation, human review
Hidden instruction in an image	OCR, then indirect prompt-injection detection with prompt shields
Prohibited symbol or brand misuse	Multimodal model or image analyzer that checks your rules, review workflow
Missing disclosure	Watermark or “AI-generated” label
Inaccurate visual claim	Answers only from what is visible, human review, abstention
Sensitive person or attribute inference	Minimize collection and refuse unsupported inference

Layer the controls: model choice → input screening → prompt protections → output screening → user disclosure → monitoring.

In the Microsoft Learn path: prompt shields and responsible AI are covered; this checklist combines them into visual policy controls for the exam.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Azure Content Understanding

glasspaper



© Rick Beerendonk

ContentUnderstandingClient

[EXAM]

Package: `azure-ai-contentunderstanding`; **API version:** `2025-11-01`

Prerequisite: Foundry resource must have these deployed:

- `gpt-5.2`
- `text-embedding-3-large`

```python
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.ai.contentunderstanding.models import AnalysisInput
from azure.core.credentials import AzureKeyCredential

client = ContentUnderstandingClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY),
    api_version="2025-11-01"
)

poller = client.begin_analyze(
    analyzer_id="prebuilt-invoice",
    inputs=[AnalysisInput(data=open("invoice.pdf", "rb").read())],
)
result = poller.result()
print(result.contents[0].markdown)          # full content as markdown
print(result.contents[0].fields)            # extracted fields dict
```


© Rick Beerendonk

Prebuilt Analyzers

[EXAM]

Analyzer	Purpose
`prebuilt-imageSearch`	One-paragraph description of an image
`prebuilt-receipt`	Merchant, items, totals
`prebuilt-invoice`	Vendor, line items, tax, totals
`prebuilt-idDocument`	Passport, driver's license
`prebuilt-documentSearch`	Semantic Markdown optimized for RAG indexing
`prebuilt-layout`	Tables, structure, QR codes, spatial layout

Supported media: Images, documents, audio, video

Choosing between the document analyzers:

Requirement	Analyzer
Tables, QR codes, and where things sit on a page	`prebuilt-layout`
Search-ready semantic Markdown for a RAG pipeline	`prebuilt-documentSearch`
Just the text	`prebuilt-read`
Known invoice fields	`prebuilt-invoice`

© Rick Beerendonk

Single-File and Multi-File Tasks

[EXAM]

Task type decides how many documents the analyzer can reason over at once.

Task	Mode	Reasoning
Single-file	**standard**	One document analyzed in isolation — high volume, low cost
Multi-file	**pro**	Several documents **plus reference data**, compared together

```
   Single-file / standard      each invoice ➡ fields

 Multi-file / pro            invoice + purchase order
                                + vendor contracts (reference data)
                             ➡ validated, reconciled result
```

Use pro mode when the requirement is cross-document: matching an invoice to its purchase order, or validating it against contract terms. Standard mode cannot compare documents, so it cannot answer "does this invoice agree with the PO?"

Both can run in one solution: bulk extraction in single-file standard, exception validation in multi-file pro.

Where the modes live: standard and pro are modes of the `2025-05-01-preview` API. The generally available `2025-11-01` API has no mode setting and analyzes one file per request; `2026-06-01-preview` adds an agentic mode (`config.workflow: "agentic"`) for multistep reasoning over one file.

glasspaper oblicum

© Rick Beerendonk

Field Schema: Extraction Methods

[EXAM]

Field extraction methods:

Method	Description	Example
`extract`	Pull value as-is, documents	`InvoiceTotal: "€1,245.00"`
`classify`	Categorize into enum values	`DocumentType: "Invoice" | "Receipt"`
`generate`	Derive via AI reasoning	Summary: 2-sentence description

Generated descriptions — for example a color scheme for a video segment — need field type **`string`** with method **`generate`**. `extract` and `classify` read or label existing content, and a non-string type cannot hold the generated text.

glasspaper oblicum

© Rick Beerendonk

Field Confidence and Thresholds

[EXAM]

Confidence thresholds — guidance for your own routing, document analyzers only:

Score range	Action
≥ 0.9	Automate (no review needed)
0.7 – 0.9	Route to human review
< 0.7	Manual processing

Field properties: `valueString`, `confidence`, `source` (bounding polygon for grounding)

For scanned troubleshooting documents, enable `estimateFieldSourceAndConfidence` to return per-field confidence and source location for review.

© Rick Beerendonk

Custom Analyzers

[EXAM]

Create via SDK:

```
poller = client.begin_create_analyzer(
    analyzer_id="expense-report-analyzer",
    resource={
        "description": "Extracts expense report totals and categories",
        "baseAnalyzerId": "prebuilt-document",
        "config": {"estimateFieldSourceAndConfidence": True},
        "fieldSchema": {
            "fields": {
                "ExpenseCategory": {"type": "string", "method": "classify",
                                    "enum": ["Travel", "Meals", "Software", "Other"]},
                "TotalAmount": {"type": "number", "method": "extract"},
                "Justification": {"type": "string", "method": "generate"},
            }
        },
        "models": {"completion": "gpt-5.2", "embedding": "text-embedding-3-large"},
    }
)
```

`baseAnalyzerId`: `prebuilt-document`, `prebuilt-image`, `prebuilt-audio`, or `prebuilt-video`

Create via REST: `PUT {endpoint}/contentunderstanding/analyzers/{name}?api-version=2025-11-01`
Poll result via `Operation-Location` response header

glasspaper oblicum

© Rick Beerendonk

Content Understanding: Build vs. Consume

Two complementary workflows:

Workflow	Purpose	Typical tool
Build an analyzer	Define fields and extraction/generation methods	Content Understanding Studio, SDK, or REST
Consume an analyzer	Submit files and process structured results	Python SDK or REST client application

Prerequisites:

- A Microsoft Foundry resource or project
- The resource endpoint and an API key, or Microsoft Entra ID authentication
- `gpt-5.2` completion deployment
- `text-embedding-3-large` embedding deployment
- Python 3.9+ for the Python SDK

Studio is useful for: visually creating schemas, testing sample content, and inspecting extracted fields before moving the definition into code.

© Rick Beerendonk

Content Understanding Client Workflow

```python
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.ai.contentunderstanding.models import AnalysisInput
from azure.core.credentials import AzureKeyCredential

client = ContentUnderstandingClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY),
    api_version="2025-11-01",
)

poller = client.begin_analyze(
    analyzer_id="expense-report-analyzer",
    inputs=[AnalysisInput(data=open("expense.pdf", "rb").read())],
)
result = poller.result()

for content in result.contents:
    print(content.markdown)
    print(content.fields)
```

REST alternative: submit the file to the analyzer endpoint, poll the returned operation URL, then process the JSON result. Use REST when the client language has no SDK or when direct HTTP control is required.

Result handling: preserve the extracted value, confidence, and source grounding so low-confidence fields can be routed to human review.

glasspaper oblicum

© Rick Beerendonk

When the Answer Is Content Understanding

[EXAM]

Reach for an analyzer when the requirement combines **structure**, **more than one modality**, and **no model training**.

Requirement in the scenario	Why Content Understanding
Preserve structure across mixed-format documents	An analyzer returns a schema, not free-form prose
Multipage tables **without training a model**	Prebuilt reasoning, no labeled dataset needed
Judge visual **and** textual evidence together	Multimodal analysis in one pass
Page citations with bounding polygons	Grounding metadata is returned with the fields
Named business fields from **varied layouts**	A custom analyzer generalizes beyond one template

Rejected alternative	Why
Azure Language text analysis	Text only — no layout, no tables
Chat completions or multimodal Responses	Free-form output, not structure-aware extraction
An Azure Machine Learning model	Requires building and training your own model
Document Layout / Document Extraction	Layout only, or no multimodal citation metadata
GenAI Prompt	Generates content instead of extracting it
`prebuilt-layout`	Returns structure, but not named business fields

© Rick Beerendonk

Video and Region-Level Analysis

[EXAM]

For video, process the right unit of content:

1. Identify the relevant time range
2. Use a video analyzer (`prebuilt-video`, `prebuilt-videoSearch`), or sample frames for a multimodal model
3. Detect speech, objects, scene changes, and actions
4. Preserve the timestamps of every segment (`startTimeMs`, `endTimeMs`)
5. Summarize or extract fields with source references

For images, region-level results may include:

- Objects, components, or defects listed as fields by an image analyzer
- Text location from OCR
- A field value grounded to a page or region with a bounding polygon

Exam distinction: a generative model can describe what it sees, but an analyzer should return structured fields and evidence when the application needs them.

© Rick Beerendonk

Visual Evidence and Accessibility

- **Alt-text:** concise, purposeful, and validated in context
- **Document fields:** preserve the source region, confidence, and page reference
- **Video:** retain timestamps, speakers, objects, and segment evidence
- **Safety:** screen visual content and protect against indirect prompt injection

Use a multimodal model for prose, and an extraction workflow when automation needs evidence.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 6

Pause here and complete Lab 6, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Azure Document Intelligence

glasspaper



© Rick Beerendonk

Document Intelligence Models

[EXAM]

Document Analysis:

Model	Extracts
`read`	Printed and handwritten text, with its location
`layout`	Text + tables + selection marks + structure

Prebuilt: Invoice, Receipt, Bank statement, W-2, 1040, ID document, Health insurance card, and more

Custom extraction models:

Type	Use Case
Custom Template	Fixed-layout forms
Custom Neural	Variable-layout documents

Add-ons for both: language detection and key-value pairs

Input limits: ≤ 500 MB, 50×50 to 10,000×10,000 px; no password-protected files

glasspaper oblicum

© Rick Beerendonk

DocumentIntelligenceClient

```python
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.ai.documentintelligence.models import AnalyzeDocumentRequest
from azure.core.credentials import AzureKeyCredential

client = DocumentIntelligenceClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY)
)

poller = client.begin_analyze_document(
    "prebuilt-invoice",
    AnalyzeDocumentRequest(url_source="https://example.com/invoice.pdf")
)
result = poller.result()

for doc in result.documents:
    for name, field in doc.fields.items():
        print(f"{name}: {field.content} (confidence: {field.confidence:.2f})")
```

Binary input: `begin_analyze_document(model_id, body=f)` with an open file

Package: `azure-ai-documentintelligence` (formerly `azure-ai-formrecognizer`)

Studio: `documentintelligence.ai.azure.com` — label training data; test prebuilt models

glasspaper

oblicum

© Rick Beerendonk

Layout: Markdown Output and Page Range

[EXAM]

Use case: Extract structured text preserving layout — for RAG chunking, search indexing

SDK (v4): `azure-ai-documentintelligence` → `DocumentIntelligenceClient`

```python
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.ai.documentintelligence.models import (
    AnalyzeDocumentRequest, DocumentContentFormat
)
from azure.core.credentials import AzureKeyCredential

client = DocumentIntelligenceClient(ENDPOINT, AzureKeyCredential(KEY))

# Analyze pages 1-3 only; output as Markdown
poller = client.begin_analyze_document(
    "prebuilt-layout",
    AnalyzeDocumentRequest(url_source="https://example.com/report.pdf"),
    pages="1-3",
    output_content_format=DocumentContentFormat.MARKDOWN,
)
result = poller.result()

# result.content = full Markdown string
```

`pages="1-3"` — analyze a page range from a multi-page PDF/TIFF
`output_content_format=DocumentContentFormat.MARKDOWN` — tables rendered as HTML markdown; headings, sections preserved

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 7

Pause here and complete Lab 7, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Page Chunks and Advanced Data Parsing

[EXAM]

```python
# Chunk by page:
for page in result.pages:
    print(f"--- Page {page.page_number} ---")
    for line in page.lines:
        print(line.content)
```

`prebuilt-layout` **with Markdown output is what "advanced data parsing" means** for RAG ingestion: layout-aware chunks that keep tables, headings, and page numbers. Reingesting scanned PDFs this way fixes retrieval that basic parsing broke.

Ingestion approach	Result
Advanced data parsing (layout + Markdown)	Tables, headings, and page metadata survive
Basic parsing + fixed-size chunks	Table structure and page numbers lost
One chunk per page	Too coarse to retrieve a single table row

Parameter trap: `output=figures` returns figure data (SDK: `output=[AnalyzeOutputOption.FIGURES]`), `content=markdown` is not a supported parameter, and raising the confidence threshold filters results rather than changing the output format.

© Rick Beerendonk

Document Intelligence vs. Content Understanding

[EXAM]

Requirement	Prefer
Read printed or handwritten text	Document Intelligence `read`
Tables, layout, selection marks, key-value pairs	Document Intelligence `layout`
Invoice, receipt, ID, or other prebuilt fields	Document Intelligence prebuilt model
Fixed or variable custom document forms	Document Intelligence custom model
Multimodal fields across documents, images, audio, and video	Content Understanding analyzer
Extract, classify, or generate custom fields	Content Understanding field schema

Confidence is not correctness: route uncertain fields to review and retain source grounding such as page, polygon, timestamp, or document reference.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 8

Pause here and complete Lab 8, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Azure AI Search

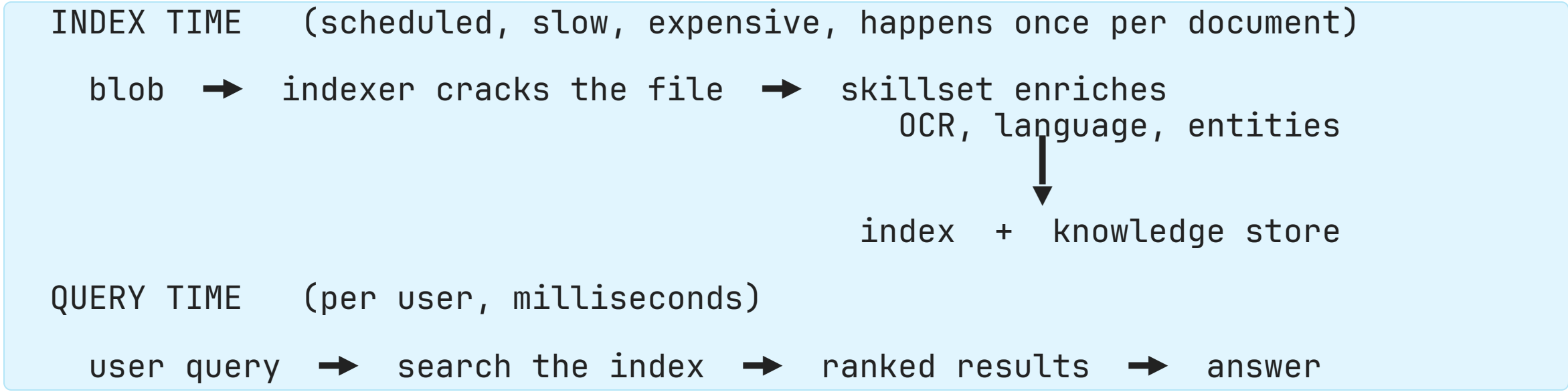
glasspaper

oblicum

© Rick Beerendonk

Two Timelines, Not One

The most common confusion with AI Search: enrichment does **not** happen when a user searches.



Consequence: adding a skill changes nothing until you re-run the indexer. Everything a user can search for had to be computed at index time.

glasspaper oblicum

© Rick Beerendonk

Enrichment Pipeline

[EXAM]

```
Data Source → Indexer → Skillset → Index ⟶ Knowledge Store
                                      ↓
                                    Search
```

Components:

Component	Purpose
Data source	Blob Storage, SQL Database, Cosmos DB, OneLake
Indexer	Cracks documents; orchestrates enrichment; runs on schedule or on-demand
Skillset	AI enrichment pipeline; chains built-in + custom skills
Index	Searchable store with typed, filterable/sortable fields
Knowledge store	Side-output of enriched projections (JSON, tables, images)

The enriched document during indexing:

- `metadata_storage_name` and other metadata (file name, URL, type)
- `content` (extracted text)
- `normalized_images` (decoded images for OCR/captioning)
- `language`, `merged_content` (added by skills: detected language, text + OCR combined)

For embedded PDF images that must be OCR-ready while retaining source citations, extract them into the indexer's `normalized_images` collection.

glasspaper oblicum
© Rick Beerendonk

Built-in AI Skills (Foundry Tools)

[EXAM]

Requires: a Foundry resource on the skillset for billing — same region when attached with a key

Skill	Output
Language detection	`languageCode`
Entity extraction	People, organizations, locations, dates, quantities
Key phrase extraction	Main topics
Translation	Translated content
PII detection	Redacted or flagged sensitive text
OCR	`text` from images within documents
Image Analysis	`caption`, `tags`; its service retires 2028-09-25
GenAI Prompt	Model-written description of an image or chunk
Azure Content Understanding	Markdown with tables and figures, already chunked
Text Split	Chunks long documents into passages
Azure OpenAI Embedding	Vectors for each chunk

Free allowance: 20 documents per indexer per day when no resource is attached

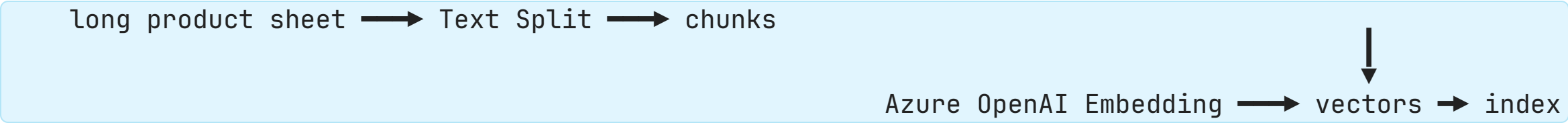
glasspaper oblicum

© Rick Beerendonk

Skills for Vector Retrieval and Scanned Text

[EXAM]

For semantic and vector retrieval you need exactly two skills:



Entity extraction, language detection, and key phrase extraction enrich text but produce **no vectors**; Merge combines fields rather than chunking.

For scanned invoices, the skill that makes the text searchable is OCR. Text Split chunks text that already exists, Translation converts language, and Image Analysis describes visual content instead of extracting the invoice text.

glasspaper oblicum

© Rick Beerendonk

Build a Knowledge Mining Solution

Typical Azure AI Search workflow:

1. Store source documents in Azure Blob Storage
2. Create a data source that connects the storage account
3. Define an index with searchable, filterable, and vector fields
4. Add a skillset for OCR, language, entities, key phrases, PII, or captions
5. Create an indexer to crack, enrich, and load the documents
6. Query with keyword, semantic, vector, or hybrid search

Hybrid search is usually the best default for RAG: lexical matching finds exact terms while vector search captures semantic similarity.

Grounding loop: search → select relevant chunks → provide citations and context → generate a response → evaluate groundedness.

© Rick Beerendonk

Choosing a Retrieval Mode

[EXAM]

Requirement	Mode
Search **private** product sheets at all	**Azure AI Search**
Exact lexical matching of terms	Full-text search (BM25)
Match a question to differently worded source text	**Vector search**
Exact codes **and** natural-language descriptions	**Hybrid search**
Reorder results that were already retrieved	Semantic ranking
Autocomplete while the user types	Suggesters
Tokenize and normalize text for lexical matching	Analyzers

Traps: semantic ranking only reranks what retrieval already returned, keyword-only search misses paraphrases, and vector-only search weakens exact code matching. Bing queries the public web, Translator converts languages, and Document Intelligence extracts content — none of them provide a private search index.

© Rick Beerendonk

Knowledge Store

[EXAM]

Persists enriched content outside the search index — for downstream analytics or export

Projection types:

Type	Use Case
Object projections	Full enriched JSON document per file
Table projections	Normalized → queryable in Excel, Power BI
File projections	Extracted images → stored as blobs

Why knowledge store? Enriched data survives index rebuilds; enables custom analytics pipelines; images can be exported separately from text content

Choosing a projection: unstructured enriched JSON → **object** projection; scanned text destined for rows and analytics → **table** projection. Reversing them stores tabular rows as loose JSON objects and forces unstructured content into a relational shape it does not fit.

© Rick Beerendonk

What a Production Index Also Needs

[EXAM]

Beyond the pipeline and the retrieval mode, plan for:

- A document key and stable source metadata
- Searchable, filterable, sortable, and vector fields as needed
- Chunking and overlap appropriate to the content
- Indexer failure monitoring and incremental refresh strategy
- Relevance evaluation with representative queries

© Rick Beerendonk

Search Security Operations

[EXAM]

Document-level filtering — three steps, all required:

1. Add the **allowed groups** to every indexed document as a filterable field
2. Retrieve the signed-in user's **group memberships**
3. Pass those groups as a **filter** on each search request

```
filter=groups/any(g: search.in(g, 'sales,finance'))
```

One index per group does not scale, an Entra access token authenticates but does not filter documents, and fetching *all* groups returns memberships that are not the user's.

Rotating a compromised admin key — order matters, or the app goes down:

```
1  point the app at the SECONDARY admin key   app keeps running
2  regenerate the PRIMARY key                 compromised key dies
3  move the app back to the NEW primary key
```

Regenerating the key the app is currently using breaks it immediately.

© Rick Beerendonk

Extraction and Search Operations

- Use `prebuilt-layout` for document structure, tables, and Markdown
- Keep embedded figures with `AnalyzeOutputOption.FIGURES`
- Use object, table, or file projections for the required downstream shape
- Use hybrid search when lexical and semantic evidence both matter
- Filter document results by the user's allowed groups

© Rick Beerendonk

Retrieval Practice — Lab 9

Pause here and complete Lab 9, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Key Takeaways

1. **Output determines service** — caption, alt-text, fields, and search are different tasks; a model answers, an analyzer returns the same fields every time
2. **Responses API** — instructions in role `"developer"`, image in the `"user"` message as `"input_image"`; result `response.output_text`
3. **ChatCompletions API** — role `"system"`, type `"image_url"` (nested `{"url": ...}`); **Phi-4 only supports this**
4. **Image generation** — `client.images.generate(model=, prompt=, n=, size=)` → `data[0].b64_json`; `images.edit` with a mask repaints one area
5. **Sora video** — create → poll → download; `seconds` as string; 24-hour expiry; max 2 concurrent; remix for one targeted change
6. **Visual safety needs layers** — image moderation, OCR plus prompt shields, policy rules, disclosure, and review
7. **Accessibility is a quality requirement** — validate generated alt-text in context
8. **Content Understanding** — `ContentUnderstandingClient`, `begin_analyze()`, `AnalysisInput(data=bytes)`; field methods: extract/classify/generate
9. **Confidence and grounding** — for document fields only; route by your own thresholds (guidance: 0.9 and 0.7); video segments carry timestamps
10. **Document Intelligence** — `read`, `layout`, prebuilt, custom template/neural; `DocumentIntelligenceClient`; ≤500 MB
11. **Choose extraction deliberately** — Document Intelligence for document structure; Content Understanding for multimodal analyzers
12. **AI Search** — indexer + skillset enrichment pipeline; knowledge store for persisted projections; built-in skills require a Foundry resource for billing
13. **Retrieval mode** — vector for paraphrase, hybrid for codes **and** prose, semantic ranking only reranks
14. **Projections** — object for unstructured JSON, table for rows and analytics, file for extracted images
15. **Search operations matter** — monitor indexing, refreshes, failures, and relevance, not just query syntax

© Rick Beerendonk

Appendix: Other Exam Areas

Translation, voice, and workflow facts that belong to other lectures

glasspaper oblicum

© Rick Beerendonk

Translation Beyond Text

[EXAM]

Scenario	Service or workflow
Text translation	Azure Translator `translate()`
Script conversion	Translator `transliterate()`
Speech translation	Speech `TranslationRecognizer`
Document translation	Translator document translation workflow with source and target storage
Multilingual RAG	Detect language, retrieve language-appropriate sources, translate only where needed

Design choice: translating before retrieval may improve cross-language search, while translating after retrieval may preserve the original source for citations. Test both with the target dataset.

In the Microsoft Learn path: text translation, transliteration, and speech translation are covered directly; document translation gets less attention.

© Rick Beerendonk

Cross-Topic Exam Controls

Some exam scenarios combine topics.

- **Low-latency voice agents:** streaming audio, server-side VAD, and `silence_duration_ms` reduce turn latency
- **Barge-in:** `input_audio_buffer.speech_started` signals that the user started speaking
- **Workflows:** `Not(IsBlank(Local.Var1))` tests that a scoped variable has a value

© Rick Beerendonk

Retrieval Practice — Lab 10

Pause here and complete Lab 10, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Appendix: This Lecture and Microsoft Learn

What this lecture adds to the Microsoft Learn path

glasspaper



© Rick Beerendonk

What This Lecture Adds to the Microsoft Learn Path

Topic	In the Microsoft Learn path	Added by this lecture
Multimodal prompts and visual grounding	A dedicated vision lesson	The request shapes of both APIs side by side
Image and video generation and editing	Dedicated lessons	Masks, product fidelity, and remix as exam decisions
Alt-text and accessibility descriptions	Accessibility is only mentioned	Alt-text as a workflow with rules and validation
Video segments and regions in images	Content Understanding is covered	How to keep timestamps, regions, and evidence
Visual policy controls and watermarking	No lesson	The risks, their controls, and how to layer them
Document Intelligence and Content Understanding	Several detailed lessons	Tables for choosing between the two
Search indexing, enrichment, and RAG	A dedicated AI Search lesson	Retrieval modes, projections, and security operations
Document translation	Text and speech translation, transliteration	The document translation workflow

In short: the path covers most core services directly; this lecture adds the exam's policy, accessibility, region-level, and operational expectations.

© Rick Beerendonk